

Does Chain-of-Thought Supervision Improve Compositional Visual Question Answering?

A Controlled Study with BLIP-2 on CLEVR

Niket Pathak Srivatsa Kamballa

Department of Computer Science, University of Illinois Chicago

{npath5, skamb10}@uic.edu

Abstract

Chain-of-Thought (CoT) supervision performs really well on text-based reasoning tasks, but effectiveness of CoT in small vision-language models over compositional visual reasoning needs some exploration. In the presented work, we conducted controlled experiment over CLEVR visual question answering benchmark and compared three setups 1) BLIP-2 baseline is a zero-shot pretrained model with no fine-tuning 2) answer-only supervised fine-tuning 3) chain-of-thought supervised fine-tuning, in which we explicitly trained base model to generate step-by-step reasoning flow which we use functional programs derived from CLEVR’s dataset before predicting the answer. We use BLIP-2 with OPT-2.7B as our starting model and use LoRA technique to train on 50,000 samples. Main results of our work show that answer-only fine-tuning had **45.95%** accuracy where as CoT supervision reached **28.90%** on a 2,000-sample validation set, both well above the zero-shot baseline of 8.75%. Our analysis revealed the following findings: CoT actually *beats* answer-only when we have short and simple questions (51.43% vs. 34.29% for program depth ≤ 4), but its performance degrades on long, multi-step chains. We think that a 2.7B parameter model does not have the required capabilities to maintain accurate visual grounding across 15–20 reasoning steps. The main take-away of our work is that CoT supervision is most useful when model have capacity to generate good reasoning steps.

1 Introduction

Visual Question Answering (VQA) combines both computer vision and natural language processing, and a model must analyze image, process a question, and generate correct answer. This does sound simple, but it gets quite complex when we have a question that requires chaining together multiple observations (“Are there more

red spheres than blue cubes to the left of the large metal object?”). The purpose of CLEVR benchmark [Johnson et al. \[2017\]](#) was compositional reasoning, with synthetic 3D scenes and questions for which it might needs upto 20 logical steps to answer correctly.

Chain-of-Thought (CoT) prompting is one of the most popular technique to emerge from the NLP community in past few years [Wei et al. \[2022\]](#), In which we ask the model to generate intermediate reasoning steps rather than directly producing output. This technique performed really well on various math word problems and tasks that requires commonsense reasoning. The core idea is very elegant: generating intermediate steps forces the models reasoning to stay on correct path with previous step serving context for next.

The question of interest is that can we use same idea in vision-language models, especially in case of smaller ones in the 2–3B parameter range. Majority of CoT work is conducted on text-only benchmarks large model ranging in 70B+. Tuning a small VLM to generate a suitable 15-step reasoning chain while keeping it grounded in the image is a hard ask.

We wanted to answer this question using direct test. We have kept our setup deliberately simple: take the same model (BLIP-2 with OPT-2.7B), fine-tune it two ways on the same 50,000 training samples 1) answer-only supervision 2) full chain-of-thought supervision and use same validation set for comparing performance. We have also included a zero-shot baseline to analyze what pretrained model can do without any fine-tuning at all.

The CoT steps are derived from CLEVR’s functional programs. In CLEVR dataset every question has a symbolic execution tree describing corrected steps needed to answer it. We transform these into natural language steps to make a clean, deterministic, noise-free reasoning step without needing a larger teacher model.

Our main findings are:

- Fine-tuning matters enormously: the zero-shot

model performed poorly with only 8.75%, while answer-only fine-tuning reaches 45.95% , a gain of over 37 percentage points.

- CoT supervision underperforms answer-only overall (28.90% vs. 45.95%), but this masks an interesting split: CoT is better on simple questions and worse on complex ones.
- The primary failure mode of CoT is visual hallucination the model generates plausible-looking reasoning steps but fills in wrong attribute values.
- We hit and fixed a subtle but critical tokenization bug in BLIP-2+OPT involving BOS/EOS token collision that caused models to stop generating after a single token.

2 Related Work

Visual Question Answering and CLEVR.. The VQA task was formally introduced by [Antol et al. \[2015\]](#), large-scale dataset with real images paired along crowd-sourced questions. This was quite popular but real-image VQA datasets are often criticized for having dataset biases that are exploited by the models [Johnson et al. \[2017\]](#). CLEVR resolves this issue as it generates questions programmatically using symbolic scene graphs and functional programs. This design choice matters for our work as functional programs are our ground-truth reasoning steps for free — it enables us to use CoT supervision deterministically without need of annotation or LLM distillation.

Vision-Language Models.. The development have led to large pretrained VLMs. BLIP-2 [Li et al. \[2023\]](#) introduced the Querying Transformer (Q-Former), a trainable bridge module that connects a frozen vision encoder to a frozen language model. It supports strong zero-shot VQA without the need of full model training. LLaVA [Liu et al. \[2023\]](#) and InstructBLIP [Dai et al. \[2023\]](#) improved it by using instruction-following fine-tuning. Even though they are capable, these models are large and expensive. We choose BLIP-2 with OPT-2.7B because it is open-source, HuggingFace compatible, and can fit on single GPU when fine-tuned with LoRA.

Chain-of-Thought Reasoning.. [Wei et al. \[2022\]](#) showed that that prompting GPT-3 with intermediate reasoning steps improves performance on arithmetic and symbolic reasoning benchmarks. [Kojima et al. \[2022\]](#) used simple instruction “Let’s think step by step” to generate useful reasoning from large models without any few-shot examples. These findings made CoT-style training and prompting quite popular. On the training front, [Ho et al. \[2023\]](#) demonstrate that CoT supervision generated by larger teacher models can improve smaller models, commonly called “reasoning distillation.” Our approach is different: we don’t use teacher model, we derive CoT steps directly from CLEVR’s functional programs, for accurate supervision.

An important gap in all of this work is that CoT works well for very large models. [Wei et al. \[2022\]](#) observed very small gains below roughly 100B parameters for text-only tasks. We hypothesize a similar threshold exists for vision-language CoT, and our experiments provide initial evidence for this.

Parameter-Efficient Fine-Tuning.. LoRA [Hu et al. \[2022\]](#) is one of the standard approach to fine-tuning large models when we have limited compute. Instead of updating all, LoRA injects small low-rank matrices into specific layers and trains only those weights. In our case, this brings the trainable parameter count from 3.7B down to roughly 3.7M which is below 0.1% of the total and it still have meaningful adaptation of the language model.

3 Problem Statement

Given an image I and a natural language question Q , the goal is simple: produce a short answer $a \in \mathcal{A}$. For CLEVR, the answer set $\mathcal{A}$ is limited but diverse. It includes yes/no responses, colors such as red, blue, green, purple, gray, cyan, brown, and yellow, shapes like cube, sphere, and cylinder, sizes including small and large, materials such as rubber and metal, and integers from 0 to 10. Altogether, this gives 28 possible answers. We set up three conditions to study the effect of supervision type:

1. **Zero-shot:** We first evaluate the pretrained BLIP-2 model directly on CLEVR, without applying any fine-tuning. This setting shows how well the model performs on the task using only its existing visual-

language knowledge, before seeing any CLEVR-specific training examples.

2. **Answer-only:** In this setup, the model is fine-tuned to predict the answer a directly from the image-question pair (I, Q) . The target is simple: just the final answer word, with no extra reasoning or explanation included.
3. **Chain-of-Thought:** Here, the model is fine-tuned to generate a step-by-step reasoning trace r before giving the final answer. This reasoning trace is built from the CLEVR functional program, so the model is guided through the visual logic of the question rather than jumping straight to the answer. The full training target is r followed by the final statement: "Therefore, the answer is: a ".

Conditions (2) and (3) are same in terms of architecture, dataset, and hyperparameters. The only difference is the type of supervision used during training. This makes the comparison controlled and easier to interpret. Our main question is whether adding reasoning supervision in condition (3) leads to better answer accuracy than the simpler answer-only setup in condition (2).

4 Technical Approach

4.1 Model Architecture

Our base model is `Salesforce/blip2-opt-2.7b`, which is built from three main components that work one after another. First, the **Vision Encoder** (ViT-L/14) processes the input image and converts it into patch-level visual feature vectors. These visual features are then sent to the **Q-Former**. It is a compact transformer that uses 32 learned query tokens to attend to the image features and compress them into 32 visual embeddings. Finally, the **Language Model** (OPT-2.7B) takes these visual tokens as a prefix, reads the tokenized question after them, and generates the answer one token at a time.

During fine-tuning, we keep both the Vision Encoder and the Q-Former completely frozen. Only the OPT language model is updated. To do this efficiently, we apply LoRA to the query and value projection matrices in each attention layer, using rank $r = 16$, $\alpha = 32$, and dropout 0.1. This gives us about 3.7 million trainable parameters out of roughly 3.7 billion total parameters, which is less than 0.1%. Freezing the visual components

keeps image feature extraction stable, and it also makes the training practical on a single consumer GPU.

4.2 Generating CoT Traces from CLEVR Programs

Each CLEVR question also comes with a functional program. This is a sequence of symbolic operations that can be executed on the scene to obtain the final answer. For example, the question "How many blue cubes are there?" maps to the program `scene` $\rightarrow$ `filter_color[blue]` $\rightarrow$ `filter_shape[cube]` $\rightarrow$ `count`.

We turn these program operations into natural language reasoning steps using a fixed template dictionary. The idea is simple here. Each symbolic operation is rewritten as short human-readable step, while still preserving original logic of the CLEVR program. A few examples of this mapping are shown below:

Program operation	Natural language step
<code>scene</code>	"Look at all objects in the scene."
<code>filter_color[blue]</code>	"Filter to keep only blue objects."
<code>filter_shape[cube]</code>	"Filter to keep only cube shaped objects."
<code>count</code>	"Count the remaining objects."
<code>unique</code>	"Identify the single matching object."
<code>relate[left]</code>	"Find objects spatially related to the current object."

The steps are numbered and concatenated into a full reasoning trace, then the answer is appended:

"Step 1: Look at all objects in the scene. Step 2: Filter to keep only blue colored objects. Step 3: Filter to keep only cube shaped objects. Step 4: Count the remaining objects. Therefore, the answer is: 2."

In case of answer-only model, same question uses only "2" as the target. The key advantage here over LLM-distilled CoT is that reasonings are deterministic and exact with no scope of no ambiguity or hallucination in supervision signal.

4.3 Answer-Weighted Loss Function

Real challenge with CoT training is that the loss is dominated by the reasoning steps. The variation could be

say CoT target is 100–150 tokens long, where as actual answer word is just 1 token. Standard cross-entropy loss does not work well as it will make model spend learning budget generating step templates right and with just little learning to get the answer correct.

To resolve the issue we are using a custom loss function that upweights the cross-entropy at the final answer token and the two tokens immediately preceding it by a factor of $5\times$. Keeping the model honest even if the reasoning steps are fluent, wrong is penalized heavily. We found this important gap in our early experiments where the model was generating appropriate-looking reasoning chains but then produced answer was wrong.

4.4 Fixing the BOS/EOS Token Collision in OPT

One big issue we faced was OPT tokenizer. In OPT, the beginning-of-sequence token and the end-of-sequence token use the same token ID, which is 2. Everything looked good at first, but was breaking generation. By default, the tokenizer adds a BOS token at the start of the target text. Generation stops immediately when model learns to output that token first, it produces token id 2 during inference. The decoder then treats it as EOS.

During initial runs it lead the CoT model to output a single character and stop. We fixed this by masking the BOS token out of the training labels by setting `labels[0] = -100`, wscuh that PyTorch’s cross-entropy loss ignores this position. We still keep the BOS token in the input sequence, since OPT was pretrained with it as a positional anchor. Removing it made generation worse. The key fix was to stop the model from learning BOS as the first target token. After that, the premature stopping problem went away completely..

5 Experimental Setup

5.1 Research Questions

This study is organized around four concrete research questions:

1. **RQ1:** Does CoT supervision improve answer accuracy over answer-only fine-tuning on compositional VQA?
2. **RQ2:** How does the benefit (or cost) of CoT supervision vary with reasoning chain length?
3. **RQ3:** What are the primary failure modes of CoT-supervised models on CLEVR?
4. **RQ4:** Is CoT supervision an effective strategy for small-capacity vision-language models (2–3B parameters), or does it require larger models to realize its benefits?

5.2 Dataset

The dataset used is CLEVR v1.0 [Johnson et al. \[2017\]](https://cs.stanford.edu/people/jcjohns/clevr/) (<https://cs.stanford.edu/people/jcjohns/clevr/>), it included 700,000 question-answer with 70,000 training scenes and 15,000 validation scenes. Scene graph generates each question programmatically, making sure of the reasoning required. Table 1 summarizes the splits we use.

Split	Full Size	Used	Avg. Depth
Train	699,989	50,000	8.4
Validation	149,991	2,000	8.4

Table 1: CLEVR v1.0 dataset statistics. We use a 50K training subset and evaluate on 2,000 validation samples.

We group the CLEVR questions into five answer types: *count*, *exist*, *query_attr*, *compare_attr*, and *compare_int*. Each group captures a different kind of reasoning, such as counting the objects, checking whether something exists, asking for an attribute, or comparing the objects. To estimate question difficulty, we use program depth, which is the number of operations in the functional program. Questions with depth ≤ 4 are labeled as “short,” while questions with depth ≥ 5 are labeled as “long.” In our 2,000-sample evaluation set, only 35 questions are short and 1,965 are long. This imbalance follows structure of the original dataset. We chose 50,000 training samples rather than the full 700,000 for practical reasons: training on the full dataset would take over a week on our hardware. 50,000 samples is a reasonable subset that covers the full distribution of question types and depths.

Our code and evaluation scripts are available at <https://github.com/Srivatsa03/Chain-of-Thought-on-CLEVR->.

5.3 Training Configuration

Table 2 summarize the hyperparameters used to training both the models. The major difference we have is lower

learning rate (1×10^{-5} vs. 5×10^{-5}) and fewer epochs (3 vs. 10) for the CoT model. This was necessary because CoT training signal is much denser. The model is not only learning the final answer. It also has to learn the structure of the reasoning template, step by step, before producing that answer. When we used a higher learning rate, the loss dropped below 0.05 by the end of epoch 1, which signalled that model is memorizing template rather than generalizing it.

Hyperparameter	Answer-Only	CoT
Epochs	10	3
Learning rate	5×10^{-5}	1×10^{-5}
Batch size	16	16
Grad. accum. steps	2	2
Warmup ratio	6%	6%
Weight decay	0.01	0.01
Precision	BF16	BF16
Answer loss weight	1.0	5.0
Max new tokens (eval)	5	200

Table 2: Hyperparameters for both fine-tuned models. LoRA rank $r = 16$, $\alpha = 32$ for both.

We train the models on an AWS EC2 `g6.xe.large` instance with an NVIDIA L40S GPU and 48GB of VRAM. We use BF16 precision instead of FP16. This choice helped us avoid the NaN gradient issues we saw earlier, especially when training on longer CoT sequences where gradients can become unstable.

5.4 Evaluation Protocol

We use same 2,000-sample validation subset to evaluate our models, and use exact string matching against lowercased ground-truth answers. For the answer-only model and the zero-shot baseline, we extract the final answer from the generated text using a priority-ordered list of valid CLEVR answers. First, we check whether the output starts with a known answer word. If it does not, we fall back to word-boundary regex matching. The CoT model is handled a little differently. Since its output is supposed to end with a sentence like “Therefore, the answer is: [answer]”, we first use a regex to look for that pattern. If that fails, we use the same priority-list method as a backup. For all models, inference is done with greedy decoding using `num_beams=1`.

6 Results

6.1 Overall Accuracy

Table 3 shows the top-level results. The zero-shot model scores only 8.75%, which is close to what you would expect from a model that has never seen CLEVR-style questions it mostly generates free-form text that doesn’t match the expected short answer format. Both fine-tuned models improve dramatically, with answer-only reaching 45.95% (+37.2 percentage points over zero-shot) and CoT reaching 28.90% (+20.2 pp). The gap between the two fine-tuned models is 17.05 percentage points in favor of the simpler approach.

Model	Setup	Acc.	Δ Zero-shot
BLIP-2	Zero-shot	8.75%	—
+LoRA	Ans.-only	45.95%	+37.2 pp
+LoRA	CoT	28.90%	+20.2 pp

Table 3: Overall accuracy on 2,000 CLEVR validation samples. Δ is the gain over zero-shot.

Figure 1 shows these results visually. The gap between answer-only and CoT is large enough to be meaningful, but the fact that both fine-tuned models leave the zero-shot model far behind tells us that fine-tuning is genuinely learning useful task-specific patterns from CLEVR.

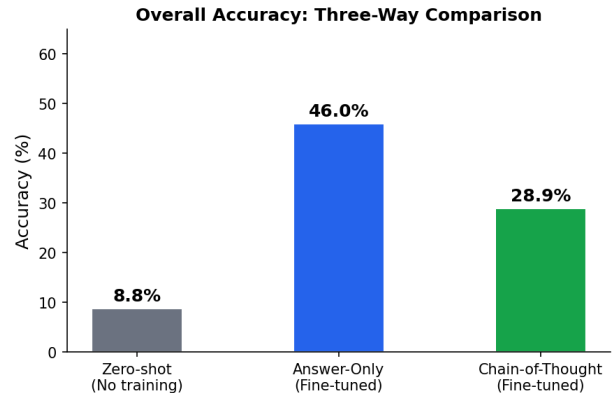


Figure 1: Overall accuracy across all three models. Fine-tuning dramatically improves over zero-shot regardless of supervision type.

6.2 Accuracy by Question Type

Table 4 breaks down accuracy across the five CLEVR question types.

Question Type	Zero-shot	Answer-only	CoT
compare_attr	11.23%	50.41%	34.38%
compare_int	28.74%	55.17%	21.84%
count	0.00%	34.90%	24.71%
exist	14.00%	61.00%	32.33%
query_attr	4.14%	36.78%	23.57%

Table 4: Accuracy by question type across all three models.

A few things stand out. First, the zero-shot model scores 0% on counting questions it never produces a bare number like “3”, instead generating full sentences. Second, the answer-only model does best on existence questions (61.00%), which makes sense since these have binary yes/no answers and the model only needs to recognize the presence or absence of an object. Third, the CoT model suffers most on *compare_int* questions (−33% relative to answer-only), which ask things like “Are there more red cubes than blue spheres?” These require two separate counting sub-programs, and the long reasoning chain gives the model more opportunities to make errors. Figure 2 shows the full breakdown visually, and Figure 3 highlights how much CoT loses relative to the answer-only baseline for each type.

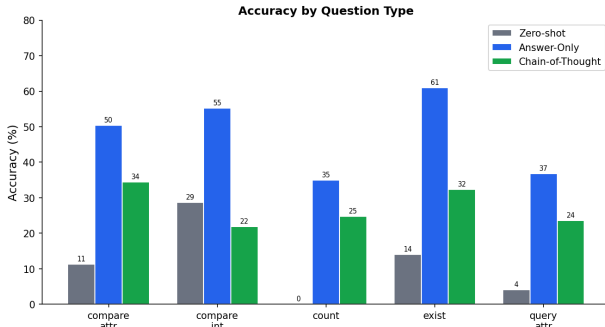


Figure 2: Accuracy by question type for all three models.

6.3 Accuracy by Reasoning Depth

The most interesting finding in our results comes from splitting by program depth. Table 5 shows accuracy for short questions (depth ≤ 4) versus long questions (depth ≥ 5).

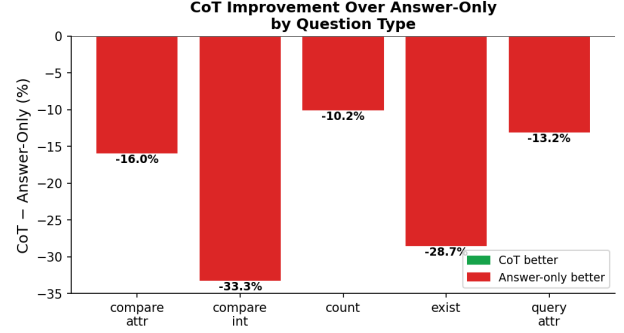


Figure 3: CoT accuracy minus answer-only accuracy per question type. All five types show a negative delta.

Depth	Zero-shot	Answer-only	CoT
Short (≤ 4 steps)	2.86%	34.29%	51.43%
Long (≥ 5 steps)	8.85%	46.16%	28.50%

Table 5: Accuracy by reasoning depth. CoT outperforms answer-only on short questions by 17 percentage points.

On short questions, CoT achieves 51.43% versus 34.29% for answer-only — a 17-point improvement. On long questions the situation completely reverses: answer-only wins by 17.66 points. This crossover is not a coincidence. Short CLEVR questions involve at most 4 reasoning steps, which is a short enough chain that a 2.7B model can generate faithfully. For these questions, having the intermediate steps in the training signal seems to genuinely help the model focus on the right attributes. But as the chain length grows to 10, 15, or even 19 steps, the model starts drifting it generates steps that are structurally correct but filled with wrong values. Figure 4 makes this crossover clear.

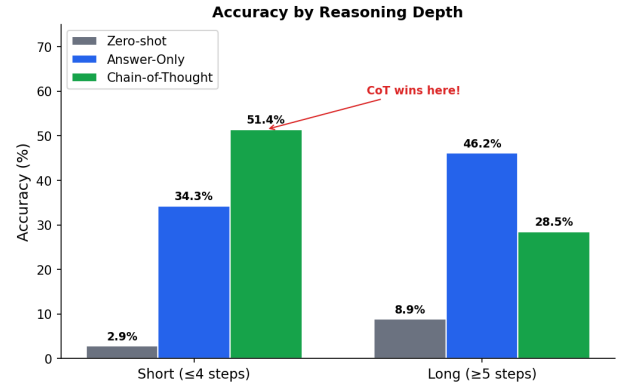


Figure 4: Accuracy by reasoning depth. The crossover between answer-only and CoT is the key finding of this study.

6.4 Training Dynamics

Table 6 and Figure 5 show how training loss evolved for both models.

Epoch	Answer-only Loss	CoT Loss
0.25	~3.1	0.730
0.50	~1.8	0.241
1.00	~1.2	0.170
2.00	~0.9	0.162
3.00	~0.7	0.159

Table 6: Training loss for both models. CoT converges faster due to structured step templates.

The CoT model converges much faster than the answer-only model its loss drops from 3.4 to 0.24 within the first half epoch. This initially looked like a good sign, but it turned out to be partly explained by the model learning the repeating structure of step templates (“Step N: Filter to keep only...”) rather than the actual reasoning. The answer-only loss starts high and decreases more gradually because each training example has a different one or two-word target with no structural patterns to exploit.

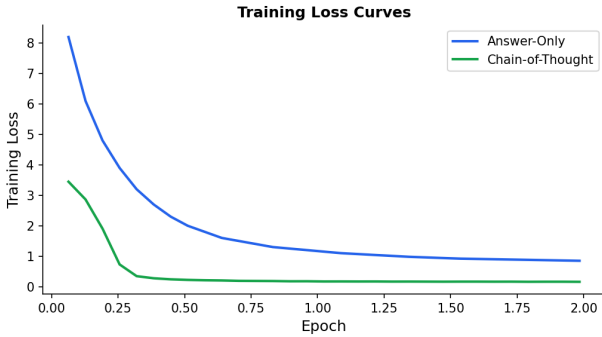


Figure 5: Training loss curves. CoT converges faster due to the predictable structure of reasoning step templates.

6.5 Error Analysis

Figure 6 summarizes where the CoT model fails. The left panel shows the most common wrong predictions: the model disproportionately outputs common answer tokens like “yes”, “no”, “0”, and “red” even when they are incorrect. This is a classic high-frequency bias when the reasoning chain breaks down, the model falls back to the most common answers it saw during training. The

right panel shows the correct/wrong split per question type, where *count* questions are the hardest for the CoT model: it gets about 75% of them wrong, and nearly all of those wrong predictions are “0” regardless of the actual answer.

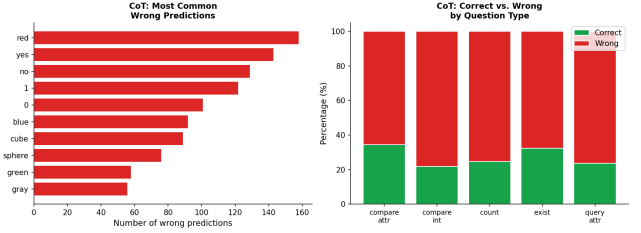


Figure 6: Left: most frequent wrong predictions for the CoT model. Right: correct vs. wrong breakdown per question type.

6.6 Qualitative Examples

Looking at individual predictions helps explain the quantitative results.

Correct CoT (depth 5):.

Q: “What is the material of the big purple object?”

GT: metal

Generated: “Step 1: Look at all objects in the scene. Step 2: Filter to keep only large sized objects. Step 3: Filter to keep only purple colored objects. Step 4: Identify the single matching object. Step 5: Identify the material of the object. Therefore, the answer is: metal.” ✓

This is an example where CoT works well: the chain is short, each step correctly narrows down the object, and the final answer is extracted cleanly.

Incorrect CoT (visual hallucination):.

Q: “What is the color of the large shiny sphere?”

GT: purple

Generated: “...Filter to keep only large sized objects. Filter to keep only metal material objects. Filter to keep only sphere shaped objects. ... Therefore, the answer is: red.” ✗

The steps look correct but the final answer is wrong, suggesting the model did not ground the color attribute in the image.

Correct Answer-only (simple binary question):.

Q: “Is there a large blue sphere?”

GT: yes

Generated: “yes” ✓

Answer-only handles binary questions cleanly with minimal output.

Degenerate CoT (complex chain):.

Q: “How many metallic objects are big blue cubes or blue objects?”

GT: 2

Generated: “Step: 0. Therefore, the answer is: 0.”

×

On very complex questions, the CoT model collapses to a degenerate one-step output.

The last example is telling. Complex union/intersection queries require the model to maintain two parallel reasoning threads simultaneously, which is beyond what the current architecture and training can handle reliably.

6.7 Comparison with Prior Work

For context, specialized models trained end-to-end on CLEVR report accuracies above 95% [Johnson et al. \[2017\]](#). Our best result of 45.95% is substantially lower, but this is expected: those models use task-specific architectures, full training data (700K samples), and end-to-end optimization. Our goal is not to match state-of-the-art CLEVR performance but to isolate the effect of CoT supervision within a fixed small VLM. The relevant comparison is between our three conditions, not against specialized systems.

6.8 Ablation Discussion

Our experimental design naturally forms a three-way ablation across supervision type. The zero-shot condition isolates the value of any fine-tuning at all: removing it costs 37 percentage points. The answer-only vs. CoT comparison isolates the effect of reasoning supervision specifically: switching from answer-only to CoT costs 17 points overall but gains 17 points on short questions. Within the CoT model, the answer-weighted loss ($5\times$ upweight on the final answer token) was critical without it, early experiments showed the model generating fluent reasoning chains but guessing answers at random. The BOS/EOS masking fix was equally load-bearing: without

it, the CoT model collapsed to single-token outputs on every question.

7 Discussion

The results paint a nuanced picture. CoT supervision is not universally helpful or harmful its effect depends strongly on how long the required reasoning chain is relative to what the model can reliably generate.

For short chains, CoT provides genuine value. The intermediate steps act as structured supervision that guides the model to attend to the right object attributes in the right order. For a question like “What material is the big purple object?”, a 5-step chain is easy enough to generate faithfully, and having that chain as a training target apparently helps the model build better representations of the visual reasoning process.

For long chains, the calculus flips. A 19-step chain requires the model to correctly identify up to 19 attribute values and spatial relationships from the image, maintain coherent state across those steps, and then land on the right answer. OPT-2.7B does not have the capacity to do this reliably. The most common failure mode is not that the model generates incoherent text it generates structurally plausible reasoning steps that happen to contain wrong values. This is arguably a harder failure mode to diagnose than garbage output, because the output looks reasonable but is incorrect.

Addressing RQ4 directly: our results suggest that CoT supervision is *conditionally* effective for small-capacity models. It works when the required reasoning chain is short enough to fit within the model’s generation capacity, but fails beyond that threshold. A 2.7B model is not inherently incompatible with CoT supervision it is incompatible with *long* CoT supervision. This is a meaningful distinction: it implies that CoT can still be useful for small models on tasks where reasoning chains are short, but should not be expected to help on tasks requiring deep multi-step chains.

We also note that the zero-shot baseline’s near-zero performance on counting questions is itself informative. The pretrained BLIP-2+OPT model was not designed for short-answer VQA and tends to generate full sentences. Fine-tuning provides the crucial adaptation to CLEVR’s output format regardless of whether CoT supervision is used.

7.1 Limitations

Several limitations of this study are worth stating explicitly. First, the model size (2.7B parameters) is small by today’s standards larger models such as LLaVA-13B or InstructBLIP-13B may handle longer reasoning chains more reliably, shifting the crossover point we observed. Second, we train on only 50,000 of the 700,000 available training examples due to compute constraints, which likely leaves accuracy headroom on the table. Third, the CoT traces are generated from rigid templates, meaning every `filter_color[blue]` always produces the same natural language step the model may be partially learning template patterns rather than visual reasoning. Finally, we use greedy decoding throughout; beam search might produce more coherent multi-step chains and is worth exploring in future work.

8 Conclusions and Future Work

We set out to answer a straightforward question: does training with explicit chain-of-thought supervision improve a vision-language model’s accuracy on compositional VQA? The short answer is: it depends. Overall, answer-only fine-tuning wins by 17 percentage points (45.95% vs. 28.90%). But that aggregate masks a real crossover: CoT is better on short, simple questions and worse on long, complex ones.

The practical implication is that CoT supervision should be matched to the model’s capacity. For a 2.7B model on the CLEVR, that capacity threshold sits somewhere around 4-5 reasoning steps. Beyond that, the benefits of explicit intermediate supervision are outweighed by the difficulty of generating a long, accurate chain. Larger models say, 7B or 13B parameters might push this threshold higher, which would be worth testing.

Several other directions follow naturally from this work. First, it would be useful to investigate *partial* CoT supervision: instead of requiring the model to generate the full reasoning chain, supervise only the last one or two steps and the answer. This might give some of the benefits of structured supervision without requiring the model to maintain state across 20 steps. Second, the data efficiency question is interesting: if we have only 1,000 training samples instead of 50,000, does CoT supervision help more? Limited data regimes are where structured supervision tends to add the most value. Finally, replacing the simple greedy decoding with beam

search might improve CoT coherence and reduce the hallucination problem, though it would slow down inference considerably.

One broader takeaway from this project is that implementation details matter a lot when working with BLIP-2. The BOS/EOS collision bug, the question padding consistency between training and inference, the FP16/BF16 precision issue any one of these could reduce accuracy to near-zero without an obvious error message. Getting these details right was as important as the experimental design itself.

9 Team Work Division and Overall Experience

Srivatsa Kamballa. took the lead on the model and training infrastructure: implementing `model.py`, `dataset.py`, and `train.py`, writing the custom `AnswerWeightedTrainer`, setting up the AWS EC2 environment, debugging all training issues including the BOS/EOS collision, the warmup steps deprecation, and the FP16 NaN gradient problem, and running all three evaluation experiments (zero-shot, answer-only, CoT). Srivatsa also led the analysis, visualization, and report writing.

Niket Pathak. was primarily responsible for the data pipeline side of the project: designing the program-to-CoT template mapping, implementing `program_to_cot.py` and `build_dataset.py`, handling the CLEVR download and preprocessing, and verifying the quality of generated reasoning traces. Niket also contributed to writing the related work section and parts of the analysis.

Overall Experience.. This project gave us real experience with the full lifecycle of a VLM fine-tuning experiment from data preparation and environment setup through training, debugging, evaluation, and interpretation. The most technically challenging part was debugging the tokenization issues in BLIP-2+OPT, which required understanding the model’s internal token handling at a low level. The most intellectually interesting part was the depth-stratified result: we initially expected CoT to either clearly win or clearly lose, and the crossover behavior was genuinely surprising. It pushed us to think more carefully about what makes CoT supervision effective and what its limits are. If we were to continue this

project, we would prioritize testing with larger models to see if the crossover point shifts.

References

- Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. [VQA: Visual Question Answering](#). In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2425–2433, 2015.
- Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Hock Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. [Instruct-BLIP: Towards General-purpose Vision-Language Models with Instruction Tuning](#). In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Namgyu Ho, Tim Salimans, Aditya Grover, Stefano Ermon, and Tim Salimans. [Large Language Models are Reasoning Teachers](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. [LoRA: Low-Rank Adaptation of Large Language Models](#). In *International Conference on Learning Representations*, 2022.
- Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C Lawrence Zitnick, and Ross Girshick. [CLEVR: A Diagnostic Dataset for Compositional Language and Elementary Visual Reasoning](#). In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2901–2910, 2017.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. [Large Language Models are Zero-Shot Reasoners](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213, 2022.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#). In *International Conference on Machine Learning*, pages 19730–19742. PMLR, 2023.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. [Visual Instruction Tuning](#). In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.